School of Electronic Engineering
and Computer Science

Programme of study:
BSc Computer Science

Project Title:
An Interactive Dashboard for the Analysis of Street-Level Crime in London

Supervisor:
Dr. Fabrizio Smeraldi

Student Name:
Ishaan Jose Cortijo Matin

Final Year
Undergraduate Project 2025/26



Date: 05/05/2026

Abstract

This project presents a web application designed to map and analyse 3.4 million London street-level crime records. Standard visualisation tools struggle with data at this scale, often causing browser lag. To solve this, the system uses a three tier architecture: an SQLite database, a FastAPI backend and a Python Dash frontend. The data was enriched with Lower Layer Super Output Area (LSOA) geographic boundaries, deprivation indices, and spatial data from OpenStreetMap to track proximity to parks and Transport for London (TfL) stations. Performance testing confirms the system is fast, with 13 out of 15 endpoints loading in under 750 milliseconds due to data caching. Analytically, the dashboard identifies clear spatial patterns: population weighted crime rates are 22% higher within 500m of TfL stations, and 11% higher near parks. The final system provides an accessible, high performance tool for environmental crime analysis.

Contents

Chapter 1: Introduction **6**
1.1 Background 6
1.2 Problem Statement 6
1.3 Aim 6
1.4 Objectives 6
1.5 Research Questions 7
1.6 Report Structure 7
Chapter 2: Literature Review **8**
2.1 Introduction 8
2.2 Domain Review: Environmental Criminology and Geovisualization 8
2.2.1 Foundational Principles of Crime Mapping 8
2.2.2 Modern Application: Environmental Factors in London 9
2.3 Technical Review: Architectures for High-Performance Visualisation 9
2.3.1 The Technical Problem: Visualising High-Density Data 9
2.3.2 Justification of the 3 Tier System Architecture 10
2.4 Summary and Identified Gap 10
Chapter 3: System Design and Requirements **11**
3.1 Requirements Analysis 11
3.2 Architecture Overview 11
Chapter 4: Implementation **13**
4.1 Data Engineering Pipeline 13
4.1.1 Data Ingestion 13
4.1.2 Data Cleaning 13
4.1.3 Data Enrichment 13
4.1.4 IMD Data Integration 14
4.1.5 Population Normalisation and Outlier Detection 14
4.1.6 TFL Station Data Import 15
4.1.7 Park Data and is_london Methodology 15
4.1.8 Database Schema 16
4.2 API Development 16
4.2.1 Performance Optimisation: Module-Level Caching 16
4.2.2 API Endpoint Specification 17
4.3 Dashboard Implementation 21
4.3.1 Interface and Visualisation Design 21
4.3.2 State Management and Technical Architecture 22
Chapter 5: Analytical Results **23**
5.1 IMD Deprivation and Crime Rate Correlation 23
5.2 Station Proximity and Crime Rates 24
5.3 Seasonal Pattern Analysis 26
5.4 Park Proximity 26
Chapter 6: Evaluation **29**
6.1 Performance Metrics 29
6.2 Visualisation Verification 30
6.3 Legal, Ethical and Sustainability Considerations 30
Chapter 7: Conclusion **32**
7.1 Summary of Work 32
7.2 Future Work 32

Chapter 1: **Introduction**

1.1 Background

The core of this project is the intersection of two distinct but related fields: the socio-spatial analysis of crime, and the technical challenge of high volume and high performance data visualisation. This project addresses the challenges associated with the accessibility and interpretability of large geospatial crime datasets. Raw data, especially at a large scale, is often slow and inconvenient for non-specialists to analyse without specific technical tools.

1.2 Problem Statement

Current analytical tools frequently obscure underlying patterns when plotting high density data points. This is known as overplotting. There is a need for a tool that handles millions of records with sub-second responsiveness for non-specialists to be able to grasp underlying correlations in crime data.

1.3 Aim

This project addresses this problem through a high performance and interactive web based dashboard. It includes filters and different ways of visualising the data so that anyone, from any technical background, can understand and spot patterns in the data.

The dashboard will then become the main tool to pair crime data with other environmental datasets to really start understanding how environmental factors affect crime rates.

1.4 Objectives

To realise the project, the following specific and measurable objectives have been set:

- **Engineer a Robust Data Pipeline**: This foundational objective involves processing and gathering raw monthly crime reports (from 2022 onwards from the Metropolitan Police Service and City of London Police) into a structured and efficient SQLite Database. This process includes the programmatic concatenation of CSV files into a dataset of over 3.4 million records. A systematic cleaning of this data was executed that removes entries with invalid or null coordinates to ensure data integrity. A data enrichment algorithm is also put in place to pre-calculate and store the corresponding London borough for each crime event to optimise query performance at later stages.

- **Develop a High-Performance API**: A decoupled backend Application Programming Interface (API) was constructed using the FastAPI framework. This API adheres to RESTful principles. It serves as an efficient intermediary to provide filtered and aggregated crime data securely and with minimal latency. By handling all this data processing on the server side, it minimises the load on the client application, therefore making it a lot quicker and ensuring responsive user experience.

- **Design and Implement an Interactive Frontend**: A user centric dashboard was created utilising Python Dash and the Plotly graphing library. Plotly was specifically selected over alternatives like Dash leaflet. This is because Plotly is superior in terms of

performance, documentation, and aesthetic capabilities for rendering large scale geospatial datasets.

- **Integrating Advanced Data Visualisations**: The system incorporates multiple, interconnected visualisations designed to offer diverse perspectives for any visitor from any technical background. This includes interactive geospatial point maps (heat maps for example), as well as summary charts and time graphs for trend analysis.

- **Conduct Exploratory Analysis**: The developed dashboard will be employed as the primary analytical tool to investigate the initial search question: Mapping and analysing how environmental factors affect street crime in London. This involves forming and visually testing hypotheses by testing out different correlations with urban features.

- **Evaluate the System**: A systematic evaluation of the final system's performance and usability was undertaken. Performance was measured by metrics such as API response times and rendering speed of millions of entries.

1.5 Research Questions

The main overarching question of this project is: How do environmental factors affect street crime in London? The title of this project describes the technique used to analyse this research question. Additional research questions can be extracted from this main one:

- RQ1: Is there a statistically significant correlation between the ONS Index of Multiple Deprivation and the rate of different crimes at the LSOA (Lower Layer Super Output Area) level?
- RQ2: Does proximity to transport infrastructure (TFL train stations) have a measurable impact on crime prevalence across London?
- RQ3: Do specific crime types exhibit significant seasonal patterns (i.e higher in summer vs. winter)?
- RQ4: Does proximity to parks have a measurable impact on crime prevalence, and does this vary by crime type?

This project answers these questions through the analytical spatial dashboard.

1.6 Report Structure

This report is divided into seven chapters. Chapter 2 provides a literature review of environmental criminology and technical architecture used in the project. Chapter 3 details the system analysis and three-tier design. Chapter 4 describes the implementation of the data pipeline and API. Chapter 5 presents the analytical results addressing all four research questions, Chapter 6 evaluates system performance, and Chapter 7 concludes with a summary of achievements and suggestions for further work.

Chapter 2: **Literature Review**

2.1 Introduction

This review summarises academic and technical literature across two key domains to establish a clear research gap. First, it examines the field of environmental criminology and the use of geovisualization to analyze spatial crime data, in order to justify the project's core research questions. Second, it reviews the technical landscape of high-performance spatial data visualisation, critiquing common methodologies and providing a benchmark-driven justification for the specific system architecture (FastAPI, Dash) as well as visualization techniques (specific maps and charts) chosen for this implementation.

2.2 Domain Review: Environmental Criminology and Geovisualization

2.2.1 Foundational Principles of Crime Mapping

Mapping crime is not a new phenomenon. As Harries (1999) notes in his foundational guide, 'Mapping Crime: Principle and Practice', the field has a long history, moving from simple pin maps to complex computational analysis. Harries establishes the core theoretical concepts that underpin this project. Indeed, this research report sets forth that crime is not random, but is a product of environmental and social conditions, requiring three ingredients: a likely offender, a suitable target, and the absence of a guardian. This provides the justification for analyzing crime patterns in relation to their environment.

Harries (1999) also categorises the primary methods of thematic mapping, such as point maps, statistical maps, and choropleth maps. This guide forms the theoretical baseline for this project's visualization choices. It explains that while point maps offer precision, they become "hopelessly confusing" in dense areas, and choropleth maps are the standard for presenting aggregated data for "neighbourhoods, or census tracts", in the United States. This is particularly relevant for this project as we will concentrate on Lower Layer Super Output Areas (LSOA) and boroughs for geographical scale, which are the United Kingdom equivalent to the latter representations.

A key theoretical framework underpinning this project's proximity analyses is Routine Activity Theory, proposed by Cohen and Felson (1979). The theory argues that crime occurs when three elements converge in time and space: a motivated offender, a suitable target, and the absence of a capable guardian. This framework directly informs RQ2 and RQ4, predicting that locations which concentrate people, like transport hubs and parks, will also concentrate opportunistic crimes, regardless of the socioeconomic character of the area.

Complementing this is Crime Pattern Theory (Brantingham & Brantingham, 1995), which introduces the concept of spatial "convergence nodes". The theory argues that specific environmental features like public parks or transport hubs, act as crime attractors and generators by routinely intersecting the daily travel paths of potential offenders and suitable targets in spaces lacking formal guardianship. This framework directly informs the spatial proximity analyses, predicting that these nodes will exhibit measurable elevations in opportunistic crime rates.

2.2.2 Modern Application: Environmental Factors in London

This project's specific focus on London crime is validated by recent, localised research. A comprehensive study by Yunus & Loo (2023) conducted an in-depth analysis of street crime in London, justifying its use as a primary case study. Their work goes beyond simple mapping to identify the specific "environmental factors" that correlate with crime patterns. Their review identifies the exact datasets this project will investigate for correlation, including "TFL Bus Stop Locations", "London Schools Atlas" and "Lower Layer Super Output Area population estimates dataset". While the ultimate goal of Yunus & Loo (2023) was to build a predictive machine learning model, their work provides an essential academic validation for this project's exploratory aim: to visually analyse the correlations between these specific environmental features and crime.

Furthermore, the visualisation of such data is a well established academic method. Harikumar et al. (2020) provide a key example of related work in their development of an "Interactive Map using Data Visualisation and Machine Learning" to display crimes against women in India. This paper (Harikumar et al., 2020) directly justifies the core method of this project by successfully employing a choropleth map to aggregate and visualise crime statistics by district, proving it is a valid and effective technique for this domain.

2.3 Technical Review: Architectures for High-Performance Visualisation

2.3.1 The Technical Problem: Visualising High-Density Data

The primary technical challenge of this project is not the volume of data (3.4 million records), but its density. Standard visualisation tools, as critiqued by Edsall & Hembree (2018) in their work on geovisualisation, fail at this scale. They note that simple dot maps become "overwhelming" and "cluttered", a phenomenon Harries (1999) also identifies.

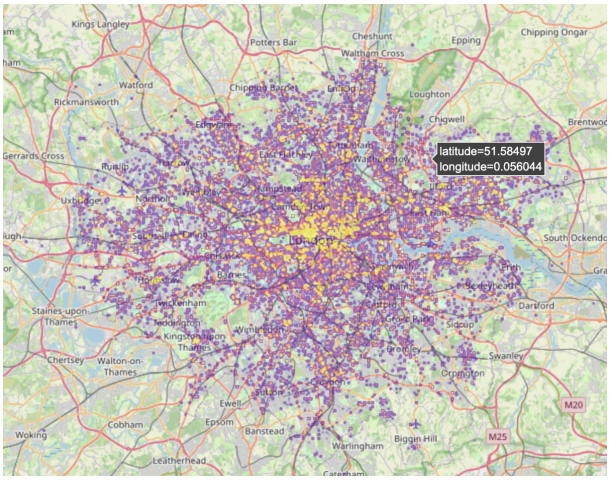


Figure 1: Heatmap generated with Plotly's 'px.density_mapbox' demonstrating the limitations of standard heatmaps for high density data

The common alternative, a two dimensional heatmap (such as Plotly's 'px.density_mapbox'), also presents significant limitations. Edsall & Hembree (2018) critique these visualisations as "ambiguous" and "difficult to understand". This is due to oversaturation, where high density areas bleed into a single, uniform "blob" of color, obscuring granular hotspots and therefore

misleading the user, as seen in Figure 1. This project's initial testing confirmed this, as shown in Figure 1. While the heatmap renders the overall distribution of crime across London, dense urban areas such as the City of Westminster and City of London collapse into a uniform region. This obscures the details and granular variations that are critical for analysis. This proves that selecting an appropriate visualisation (such as the Choropleth map) is a non-trivial research and design challenge. Further research into advanced techniques, such as the 'datashader' library, is justified as it is a technology specifically designed to solve this problem of 'overplotting' for datasets with millions of points (Datashader, 2024)

2.3.2 Justification of the 3 Tier System Architecture

To serve this data effectively, a high performance system architecture is paramount. A monolithic application, where the data processing and the data visualisation occur in the same process, would be very slow and unresponsive. This project therefore implements a decoupled three tier model (Database -> API -> Frontend) to ensure the dashboard remains lightweight.

For the frontend, Braganca & Kho (2023), in a comparative analysis of Python microframeworks, identify Dash as being "primarily used for building analytical web applications". Their study notes Dash's "superior loading speed" and "exceptional speed in loading and reloading times", justifying its selection over alternatives for a data centric dashboard.

For the backend API, performance is the most critical factor. A 2025 benchmark by Bednarz & Miosz (2025), provides a definite, quantitative justification for selecting FastAPI over other microframeworks such as Flask. Their study directly compares the performance of the project's exact stack: "FastAPI + SQLite3" versus "Flask + SQLite3". Their findings conclude that FastAPI achieves approximately 77% faster request throughput (4783 req/s vs. 2705 req/s) and is 100% faster at database reads for single records (2.5ms vs 5.0ms) (Bednarz & Miosz, 2025). This benchmarked performance is the primary technical justification for building the system's API with FastAPI, as it is essential for handling concurrent user data requests on an interactive dashboard.

2.4 Summary and Identified Gap

This literature review identifies a clear research gap. The domain review establishes what factors to analyse (Yunus & Loo, 2023), and confirms that choropleth maps are a valid method (Harikumar et al., 2020). The technical review simultaneously identifies why this is a difficult visualisation problem, as standard maps fail at high density (Edsall & Hembree, 2018) (Harries, 1999).

This project bridges this gap. Its contribution is the design and implementation of an interactive dashboard that applies a proven, high-performance, benchmarked architecture (FastAPI + Dash), as justified by Bednarz & Miosz (2025) and Braganca & Kho (2023), to solve a known, valid visualisation problem within a well established research domain. It provides a public facing tool for exploratory analysis where existing academic tools are often static (papers) or use ambiguous visualisations (heatmaps).

Chapter 3: **System Design and Requirements**

3.1 Requirements Analysis

Functional requirements include data ingestion, geospatial/temporal filtering, and aggregation. The system must ingest and validate 3.4 million crime records, enriching them with borough assignments, IMD deprivation scores, and proximity metrics to transport hubs and parks. It must support flexible querying across time periods, borough, and crime types, retiring aggregated statistics in JSON format.
Non-functional requirements focus on sub 750ms API latency and UI responsiveness. This performance target is critical for interactive exploration: above this latency, interfaces become sluggish.

3.2 Architecture Overview

The system implements a decoupled three-tier architecture (Figure 2) that separates data processing, application logic and presentation. This enables the dashboard to remain lightweight and responsive while heavy computations occur server side.

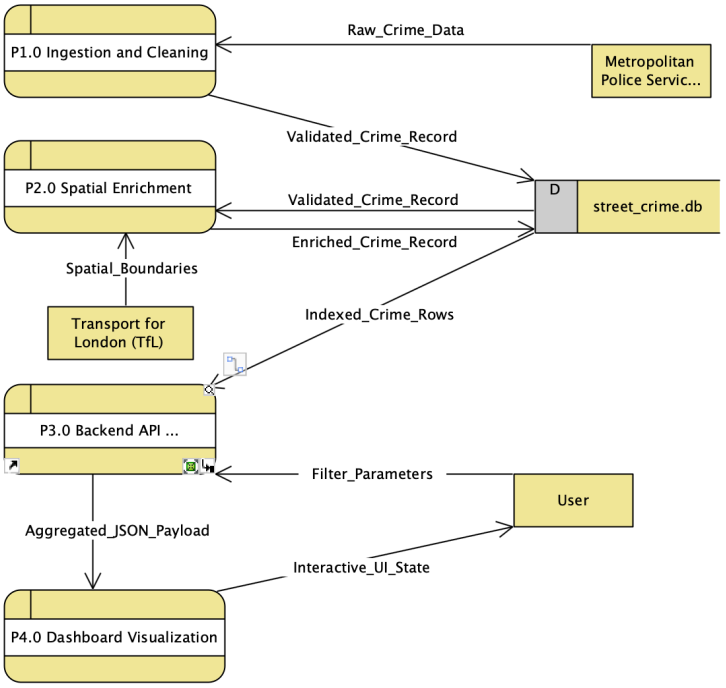
Tier 1: Data Layer (SQLite Database): The foundation is a SQLite database that stores the processed crime dataset. Raw crime records flow through a multi-stage pipeline before arriving at the database. Stage P1.0 ingests monthly CSV files from the Metropolitan Police Service and performs initial data cleaning. Stage P2.0 enriches these records with spatial data: borough data with GeoJSON polygon matching, and IMD deprivation scores via LSOA lookup tables. These records are indexed and persisted in the database.

Tier 2: Application Logic (FastAPI Backend): Stage P3.0 implements a RESTful API layer using FastAPI. This tier handles all server-sided data processing: SQL queries against the database, statistical aggregations, geospatial calculations. The API accepts filter parameters from the frontend, executes the queries, and returns JSON payloads. This ensures the client receives only pre-computed data rather than raw records, reducing computation on the browser.

Tier 3: Presentation Layer (Dash Dashboard): Stage P4.0 is a Dash-based frontend that receives JSON payloads from the API and renders interactive visualisations. The dashboard maintains client side state using Dash's dcc.Store component.

Figure 2 illustrates how raw crime data from the Metropolitan Police Service flows through the pipeline: ingestion and cleaning -> spatial enrichment -> persistent storage in SQLite -> query and aggregation -> visualisation.

Figure 2: Data Flow Diagram



Chapter 4: **Implementation**

4.1 Data Engineering Pipeline

A robust Python pipeline was engineered to handle the 3.4 million records.

- **Ingestion**: A script iterates through the directory of monthly CSVs, concatenating them into a single DataFrame.
- **Cleaning**: Records with '`null`' geospatial coordinates are dropped from the database.
- **Enrichment**: A spatial join algorithm maps every crime coordinate point to a London Borough using a GeoJSON polygon boundary file. This '`Borough`' Tag is stored in the database in a new column to speed up future queries (eg. filter by borough)

4.1.1 Data Ingestion

The ingestion pipeline is primarily implemented in `db_create.py`. A helper function, `concat_csvs(dir)`, recursively scans the MET Police download directory using `glob` and concatenates all monthly CSV files into a single unified dataset. This combined dataset is all written to `all_data.csv`, giving one consolidated file containing data across all months.

A new SQLite database is then created (`street_crime.db`). A crimes table is defined with the core analytical fields: `month`, `longitude`, `latitude`, `crime_type`, and `lsoa`. The script reads `all_data.csv` into a pandas DataFrame and renames the relevant raw CSV columns (by index) to match this schema. The cleaned result is saved as `all_data2.csv` and then loaded into the crime dataset via `to_sql`, bulk inserting all crime records into the database in one step.

4.1.2 Data Cleaning

During loading and preparation, the pipeline focuses on geospatially meaningful records. This means we only retain rows where the latitude and longitude values are not null. Any records with missing or null coordinates are treated as unusable for mapping and are removed from the database. We do this by applying the following SQL instruction to the table:

```
DELETE FROM crimes WHERE longitude IS NULL OR latitude IS NULL;
```

This ensures that subsequent spatial operations (such as mapping, clustering, or borough assignment) work on a clean set of points with valid locations.

This cleaning step means that future components used do not need to constantly guard against null values, simplifying logic in queries and improving reliability of analyses.

4.1.3 Data Enrichment

Spatial enrichment is handled by `add_borough_toDB.py`. The script connects to `street_crime.db`, reads the `ROWID`, `latitude` and `longitude` for each crime, and loads the official London borough boundaries from the `GeoJSON` file `LondonBoroughs.geojson` using `GeoPandas`. These boundaries were found on TFL.

Each crime record's latitude and longitude coordinates are converted into a Shapely point object (a geometric representation of a single coordinate in 2D space) and wrapped into a GeoDataFrame with EPSG:4326 coordinates, enabling spatial operations such as testing

whether the point falls within a borough boundary polygon. A spatial join (`sjoin`) is run between the crime points and the borough boundaries polygons with the `within` command. This algorithm assigns a borough to every crime.

After the join, the script changes the crime table to add a new borough `text` column. It then bulk-updates the table with the results of the spatial join, writing the borough name into this new field for each crime, keyed by `ROWID`. Storing the borough tag straight into the database drastically speeds up future queries and API calls that filter or aggregate by borough, because they no longer need to recompute spatial operations.

Of the 3,379,514 crime records, 3,366,968 were successfully assigned to a borough. The remaining 12,546 records, representing 0.37% of the original dataset, returned NULL. This represents crimes with coordinates that fall outside the GeoJSON polygon boundaries. These are usually near borough edges. These records are excluded from the borough-level queries via WHERE borough IS NOT NULL clause applied in the relevant API endpoints.

4.1.4 IMD Data Integration

A key objective was the enrichment of the crime dataset with the deprivation data from the ONS Index of Multiple Deprivation (IMD). The IMD assigns each Lower Layer Super Output Area (LSOA) in England an overall rank and decile score based on seven weighted domains of deprivation. These include income. Employment, health, and crime. Integrating this data enables the statistical investigation of the first research question: whether there is a significant correlation between deprivation and crime rate at the LSOA level.

This enrichment is implemented in `imd_enrich.py`. The primary source is the ONS IMD dataset `indices_of_deprivation.csv`, which contains IMD ranks and deciles keyed on 2011 LSOA boundary codes. A challenge arose from a boundary change in LSOA codes: the MET Police crime data uses 2021 LSOA codes, while the IMD dataset uses the 2011 boundary definitions. To bridge this discrepancy, a lookup table (lsoa11_to_lsoa21_lookup.csv) was used to merge each crime record's 2021 code back to its 2011 equivalent before joining to the IMD data. For LSOAs that already matched 2011 codes directly, the original code was preserved.

The enrichment procedure first adds two new columns to the crimes table: imd_rank (INT) and imd_decile (INT), using ALTER TABLE. It then performs a three way merge in pandas: crime LSOAs are joined to the 2011-2021 lookup, which produces a target_lsoa11 column. This then is joined to the IMD data on that 2011 code. The resulting rank and decile values are written to a temporary imd_sync table and then bulk applied to the main crimes table via a SQL UPDATE…FROM statement keyed by row ID.

Of the 3,379,514 total crime records, 3,379,318 were successfully enriched with IMD data: about 99.99%. The 196 unmatched records represent crimes in boundary edge cases, and are treated as NULL in subsequent statistical analysis.

4.1.5 Population Normalisation and Outlier Detection

A critical limitation of raw crime counts as an analytical metric is that they mix population density with crime prevalence. For example, a borough with twice the residents would naturally record more crimes even if it were proportionally safer. To enable spatial comparison with relevant information, a population normalisation pipeline was implemented in `population_normalise.py`. This script produced a new lsoa_stats table that stores an annualised crime rate per 1000 residents for each LSOA.

The population source is the ONS Mid-2022 LSOA 2021 population estimate (`lsoa_population_estimates.xslx`). This dataset provides a total resident population figure for each 2021 LSOA code, directly compatible with the LSOA codes that were used in the crime database. The annualised crime rate is computed with the following formula:

```
annual_crime_rate_per_10000 = (total_crime/span_years/population)*1000
```

Here, span_years is derived from the earliest and latest months present in the crimes table, evaluated as 3.0 years across July 2022 to June 2025. This ensures that the rate is normalised regardless of the date range queried.

The resulting lsoa_stats table covers 10,768 LSOAs and stores five fields per record: lsoa_code, population, total_crimes, annual_crime_rate_per_1000 and an is_outlier flag. An index on lsoa_code is created to ensure joins with the crimes table are efficient in API queries.

To implement outlier detection, two categories of LSOAs are flagged as outliers and excluded from correlation analyses:

- LSOA with a population below 500, which are usually representative of industrial zones, commercial areas etc., where population estimates are not reliable for crime calculations
- LSOAs with an annual crime rate above the 99th percentile of the distribution, which represent outliers likely caused by major transport hubs or entertainment districts.

Of the 10,768 LSOAs, 543 were flagged as outliers (5%), leaving 10,225 non-outlier LSOAs available for statistical analysis. This filtering step is critical for the validity of the Pearson correlation results presented in the next section, as including commercial outliers would artificially modify the deprivation-crime relationship.

4.1.6 TFL Station Data Import

To support the investigation of Research Question 2 (whether proximity to transport hubs has a measurable impact on crime prevalence), a dataset of TFL train station locations was imported into the database from the TFL API This is implemented in import_tfl_stations.py

The source file is a KML file sourced from the TFL Open Data portal, containing point geometries for all 302 train stations in the TFL network. The script uses GeoPandas to parse KML directly into a GeoDataFrame. It extracts each station name and derives numeric lat and lon columns for the point geometry. The resulting data is loaded into a new stations table in street_crime.db via to_sql.

After import, the stations table contains 302 records with the following schema: name, lat, lon, and a Description field.

The distance calculation between each LSOA centroid and its nearest station is identified in the next work section of the report.

4.1.7 Park Data and is_london Methodology

To facilitate the proximity analysis of Research Question 4, park geographic data was fetched using the OpenStreetMap Overpass API. Using a standard bounding box (51.28, -0.51 to 51.70, 0.33), the system successfully ingested 3893 park records. However, initial testing revealed a significant data quality issue: the bounding box captured thousands of parks from surrounding counties, including Essex, Kent, Surrey. Because the Metropolitan Police dataset contains

records from operations on the fringes of Greater London, these non-London areas were being included in the "far" groups for proximity analyses. This contamination artificially deflated the baseline crime rates, causing initial results to show exaggerated elevations, such as 3x increase for stations and 6x for parks.

To resolve this and ensure methodological rigor, a new is_london integer flag was added to the lsoa_stats table. An LSOA is defined as being within London if at least 50% of its associated crime records were successfully assigned a non null borough during the spatial join phase. This borough proxy was chosen over LSOA code prefixes because London utilises mixed prefixes that proved unreliable for filtering. By implementing this, 99.97% of LSOAs were clean, resulting in a final subset of 5158 London LSOAs, of which 4441 were classified as non outliers.

The impact of this correction was substantial, adjusting the station proximity elevation from 3x to a defensible +22%. Park proximity went from 6x to +11%. To maintain consistency in the frontend, the /parks API endpoint was modified to filter visible park markers based on the bounding box of London LSOA centroids, reducing the number of rendered parks to 2800.

Finally, precise LSOA centroids were fetched from the ArcGIS ONS endpoint via add_lsoa_centroids.py script. These coordinates were stored as lat and lon fields within the lsoa_stats table to optimise queries. Using these centroids, the system calculates and stores Haversine distance for each LSOA, populating the fields distance_to_park_m, nearest_park, near_park (0/1 flag for distance<500m), and parks_within_1km.

4.1.8 Database Schema

The SQLite database street_crime.db implements the following schema. The primary crimes table stores records with fields: month, longitude and latitude, crime_type, lsoa, borough, imd_rank, and imd_decile. Four indexes are created for query efficiency: idx_month_borough, idx_crime_type, and idx_borough.

The lsoa_stats table aggregates crime statistics at the LSOA level, storing: lsoa_code, population, total_crimes, annual_crime_rate_per_1000, is_outlier, is_london, distance_to_station_m, nearest_station, near_station, stations_within_1km, distance_to_park_m, nearest_park, near_park, parks_within_1km, and lat/lon. Two indexes support efficient lookups: idx_lsoa_stats_code and idx_lsoa_stats_is_london.

Temporary tables support the enrichment pipeline: imd_sync holds intermediate IMD rank/decile values during the bulk update process: _centroid_tmp stored LSOA centroid coordinates fetched from ArcGIS during the add_lsoa_centroids.py process. The stations table stores 302 TFL train station records (name, lat, lon, description). The parks table stores OpenStreetMap parks (id,. Name, lat, lon) with an index on coordinates for efficient spatial queries.

4.2 API Development

4.2.1 Performance Optimisation: Module-Level Caching

Six endpoints serve static analytical views that do not change with user-applied filters: /imd_scatter, /imd_deciles, /seasonal, /correlation_results, /station_analysis, and /park_analysis. These endpoints are implemented using module-level variable caching. On the first call, each endpoint executes its full computation: multi-table spatial joins, Pearson correlation calculations, aggregations. The result is then stored in a global module variable (eg. imd_deciles_cache = results, initialised at module level). On subsequent requests, the endpoint checks if the cache variable is non-NULL and returns the cached results immediately without database read.

4.2.2 API Endpoint Specification

The FastAPI backend is fully operational with the following 15 endpoints:

- `GET /crimes`: Returns raw street level crime incidents from the crimes table, with flexible filtering over time, crime type, and borough.
  - Query Parameters (all optional):
    - `start_date` (string, e.g. "`2023-01`"): Lower bound for the month field
    - `end_date` (string, e.g. "`2023-06`"): Upper bound for the month field
    - `specific_month` (string, e.g. "`2023-03`"): Exact month to filter on
    - `crime_type` (string, e.g. "`Bike Theft`"): Filters incidents to a specific crime category
    - `borough` (string, e.g. "`Wandsworth`"): Filters incidents to a specific London borough
  - Filtering Behaviour:
    - All filters are combined using logical `AND`, executed as SQL on the database
    - If no filters are provided, all records are returned
  - Response:
    - Returns JSON array of rows, each containing at least:
      - `latitude`: latitude of the crime location
      - `longitude`: longitude of the crime location
      - `crime_type`: category of the crime
      - `month`: month of the incident in `YYYY-MM` format
      - `borough`: associated London borough

- `GET /boroughs`: Returns a list of distinct London boroughs that are present in the database. This is mainly used to load filters into the dashboard.
  - Query Parameters: `None`
  - Response:
    - JSON object with a single key: `boroughs`. An array of boroughs sorted alphabetically. (e.g. `{"boroughs":["Barking","Barnet"...]}`)

- `GET /summary`: Provides a high level summary of crime counts over the selected subset of data. Returns total number of crimes and breakdown of counts by crime type. Endpoint used for summary cards, bar charts etc.
  - Query Parameters (all optional):
    - `start_date` (string, e.g. "`2023-01`"): Lower bound for the month field
    - `end_date` (string, e.g. "`2023-06`"): Upper bound for the month field
    - `crime_type` (string, e.g. "`Bike Theft`"): Filters incidents to a specific crime category
    - `borough` (string, e.g. "`Wandsworth`"): Filters incidents to a specific London borough
  - Filtering Behaviour:
    - All filters are applied using logical `AND`, similar to `/crimes`
  - Response:
    - `total_crimes` (integer): total number of records that satisfy the filter
    - `crime_counts_by_type` (dictionary): dictionary with crime types names as keys and count of each crime type as values. If a specific crime type is passed, the breakdown is skipped.

```
{
  "total_crimes": 15234,
  "crime_counts_by_type": {
    "Violence and sexual offences": 5234,
    "Anti-social behaviour": 4200,
    "Vehicle crime": 1800,
    ...
  }
}
```

- `GET /trends`: Provides time series trends of crime counts over months. It can return: total crimes per month or crimes per month broken down by crime type. This endpoint is useful for line charts showing how crime changes over time.
  - Query Parameters (all optional):
    - `start_date` (string, e.g. "`2023-01`"): Lower bound for the month field
    - `end_date` (string, e.g. "`2023-06`"): Upper bound for the month field
    - `crime_type` (string, e.g. "`Bike Theft`"): Filters incidents to a specific crime category
    - `borough` (string, e.g. "`Wandsworth`"): Filters incidents to a specific London borough
    - `breakdown_by_type (boolean, default false)`: if false, turns a single count per month (overall trend). If true, returns separate series per crime type
  - Response (overall trend, `breakdown_by_type=false`)

```
[
 {"month": "2023-01", "count": 1200},
 {"month": "2023-02", "count": 1105},
 ...
]
```

  - Response (trend by trend, `breakdown_by_type=true`)

```
{
  "Burglary": [
    {"month": "2023-01", "count": 150},
    {"month": "2023-02", "count": 140}
```

```
    ],
    "Robbery": [
      {"month": "2023-01", "count": 90},
      {"month": "2023-02", "count": 85}
    ]
  }
```

- `GET /crime_types`: Provides a list of all distinct crime types present in the dataset. Used for building crime based filters on the dashboard.
- Query Parameters: `None`
- Response: JSON Object:
  - crime_types: an array of unique crime type strings, sorted alphabetically
  - e.g:

```
{
  "crime_types": [
    "Anti-social behaviour",
    "Burglary",
    "Criminal damage and arson",
    "Drugs",
    "Robbery",
    "Vehicle crime",
    ...
  ]
}
```

- `GET /summary_by_borough`: Returns crime counts grouped by London borough, used to drive the choropleth map visualisation
  Query Parameters (all optional):
  - `start_date (string)`: lower bound for the month field
  - `end_date (string)`: upper bound for the month field
  - `crime_type (string)`: filters to specific crime category
  - `borough (string)`: filters to specific London borough

  Response: JSON array of objects, each containing:
  - `borough`: London borough name
  - `count`: total number of crimes in that borough matching the filters
  - `Eg [{"borough": "Westminster", "count": 4821}, ...]`

- `GET /imd_scatter`: Returns per_LSOA annual crime rate per 1000 residents paired with IMD rank and decile scores. Results are computed by joining the crimes table with the lsoa_stats table, filtering to LSOAs with a population of at least 500 and a non-null IMD rank, and calculating an annualised crime rate over the 3 year dataset span.
  - Query Parameters: `None`
  - Response: JSON array of objects, each containing:
    - `lsoa_code`: LSOA identifier
    - `borough`: associated London borough
    - `imd_rank`: imd rank for the LSOA (lower=more deprived)
    - `imd_decile`: imd decile (1=most deprived, 10=least deprived)
    - `annual_rate_per_1000`: annualised crime rate per 1000 residents

- `GET /imd_deciles`: Returns the mean annual crime rate per 1000 residents aggregated by IMD decile, for all non-outlier LSOAs.
  - Query Parameters: `None`
  - Response: JSON array of 10 objects, one per decile, each containing:
    - `imd_decile`: integer (1-10)
    - `avg_rate`: mean annual crime rate per 1000 residents for that decile

- `GET /correlation_results`: Returns Pearson correlation coefficients between IMD rank and annualised crime rate per 1000 residents, broken down by crime type. Only includes results where p<0.05 and n>=30 LSOAs. Results are sorted by absolute r value descending.
  - Query Parameters: `None`
  - Response: JSON array of objects, each containing:
    - `crime_type`: crime category
    - `pearson_r` : pearson correlation coefficient (rounded to 4 decimal points)
    - `p_value`: p-value of the result
    - `strength`: classification: weak, moderate or strong
    - `n_lsoas`: number of LSOAs included in the calculation

- `GET /seasonal`: Returns average monthly crime count per season per crime type, computed across the full dataset regardless of any filters. This endpoint intentionally ignores date, borough and crime type filters. Seasonal comparison is only meaningful across a complete annual cycle.
  - Query Parameters: `None`
  - Response: JSON array of objects, each containing:
    - `crime_type`: crime category
    - `season`: one of Spring, Summer, Autumn, Winter
    - `avg_monthly_count`: average number of crimes per month in that season

- `GET /proximity_dots`: Returns LSOA centroids with pre-computed crime rates and distances to nearest station and park. Filters to is_london = 1.
  - Query Parameters: None
  - Response: JSON array of objects, each containing:
    - `lsoa_code`: LSOA identifier
    - `lat`: latitude of LSOA centroid
    - `lon`: longitude of LSOA centroid
    - `annual_crime_rate_per_1000`: annualised crime rate per 1000 residents
    - `distance_to_station_m`: distance in metres to nearest TFL station
    - `distance_to_park_m`: distance in metres to nearest park

- `GET /parks`: Returns park locations filtered to the London LSOA bounding box. Used by the proximity map to overlay park markers on top of LSOA centroids.
  - Query Parameters: None
  - Response: JSON array of objects, each containing:
    - `lat`: latitude of park centre point

  - `lon`: longitude of park centre point
  - `name`: name or identifier of the park

- `GET /station_analysis`: Returns distance decay and crime type breakdown for TFL station proximity analysis. Includes weighted crime rates per distance band and per crime type near vs far from stations.
  - Query Parameters: None
  - Response: JSON object containing:
    - `decay`: array of distance bands with avg_rate and n_lsoas
    - `by_crime_type`: array with `crime_type`, `near_station` (0/1) and `avg_rate`
    - `density_bands`: array with station count bands (0-1,1-3,4-7,8+) and corresponding rates

- `GET /park_analysis`: Returns distance decay and crime type breakdown for park proximity analysis. Includes weighted crime rates per distance band and per crime type near vs far from parks.
  - Query Parameters: None
  - Response: JSON object containing:
    - `decay`: array of distance bands with avg_rate and n_lsoas
    - `by_crime_type`: array with `crime_type`, `near_park` (0/1) and `avg_rate`
    - `density_bands`: array with station count bands (0-1,1-3,4-7,8+) and corresponding rates

- `GET /station_map_data`: Returns average crime rates within a 500m radius of each TFL station for map overlay interactions.
  - Query Parameters: None
  - Response: JSON array of objects, containing:
    - `station_name`: name of the TFL station
    - `lat`: latitude coordinate
    - `lon`: longitude coordinate
    - `avg_annual_rate`: mean annual crime rate per 1000 residents within 500m

4.3 Dashboard Implementation

4.3.1 Interface and Visualisation Design

The frontend dashboard is implemented in app.py using Python Dash and the Plotly graphing library, served via a Flask development server. The dashboard follows the 3-tier architecture described in Chapter 3, communicating with the FastAPI backend only via HTTP requests. The interface is split into three main parts: a sidebar on the left for filters, a large central area with tabs for viewing maps and charts, and a permanent “Research Insights” section at the bottom.

The left sidebar holds the main controls, allowing the user to pick a date range and select specific London boroughs or crime types. The main Map tab offers three different ways to view the data. The first is a standard dot map that plots up to 20,000 individual crimes as colored dots based on the type of crime. The second is a choropleth map, which colours in the official borough boundaries based on the total number of crimes in each area. The third option is a custom built proximity map designed specifically to answer the project's spatial research questions. This map displays the center point of every London LSOA as a single dot, where the size and color of the dot represent the local crime rate. A toggle allows the user to overlay either TFL stations or parks as markers on top of these dots. This creates a visual tool for spotting where high crime areas cluster around public transport or green spaces.

The Analysis tab focuses on the data the user has filtered. In earlier versions of the project, this tab held several different analytical views, but it has now been streamlined to just two core charts to prevent the interface from feeling cluttered. Users can look at a donut chart with the “crime type split” to see the breakdown of different crime types, or a “Borough comparison” bar chart to see how total crime numbers compare across the selected boroughs.

The more complex analytical views were moved out of the Analysis tab and placed into a dedicated "Research Insights" section at the bottom of the dashboard. This section features four summary cards, one for each of the research questions. Clicking "open details” on any of the cards opens a large modal containing the full findings. For RQ1, this includes a scatter plot comparing deprivation to crime rates. RQ2 shows charts tracking how crime drops as you move away from a station. RQ3 displays a line chart tracking seasonal trends. Because tracking 14 different crime types on one chart can be visually confusing, it was decided to represent each crime type on a different chart. Finally, RQ4 contains the charts proving the link between parks and crime, including distance decay and density bands.

An important design choice in the dashboard is how it handles the sidebar filters. The dot map, choropleth map and the two charts in the Analysis tab are fully interactive and change based on the filters the user selects. However, the proximity map and the four research insight windows completely ignore these filters and always use the full three year dataset. This was done on purpose as the research questions are meant to uncover geographic patterns across the whole of London on a large timeframe. If a user filtered the data down to just a single month or one borough, the sample size would be too small, and the resulting research charts would become statistically meaningless.

4.3.2 State Management and Technical Architecture

A critical technical challenge in building Dash applications is managing the application state and preventing the backend server from being overwhelmed by redundant API calls. All communication with the FastAPI backend is handled synchronously using Python’s requests library, fetching JSON payloads that are then converted into Pandas DataFrames for plotting.

The left sidebar holds the main user controls, including a date picker and dropdowns for boroughs and crime types. To keep the dashboard running smoothly and avoid overwhelming the server, these filters do not update the data automatically as soon as they are clicked. Instead, the user clicks an “Apply Filters” button. When triggered, the backend fetches the requested data, which is then stored directly in the user’s web browser using Dash’s dcc.Store component. By caching this data client-side, the user can instantly switch between the dot map, choropleth map, and analysis charts without triggering any new network requests. Loading spinners (dcc.Loading) are used to give the user visual feedback during the brief delay while the backend processes the initial download. Because rendering millions of points would crash a web browser, the API artificially caps the dot map payload at 20000 records, using WebGL to maintain smooth panning and zooming.

Chapter 5: Analytical Results

5.1 IMD Deprivation and Crime Rate Correlation

To investigate Research Question 1 (whether a statistically significant correlation exists between the ONS Index of Multiple Deprivation and crime rate at the LSOA level), a Pearson correlation analysis was implemented across all crime types in the dataset. This analysis was implemented in imd_correlation.py, added to the backend with a new endpoint (/correlation_results).

The correlation is computed between two variables for each LSOA: the IMD rank (lower rank means greater deprivation), and the annualised crime rate per 1000 residents, calculated per crime type using the formula ((count/span_years)/population)*1000. The analysis uses the full 10,225 non-outlier LSOAs rather than the London-only subset used in RQ2 and RQ4. This is justified because Pearson correlation is scale-invariant. Therefore, LSOAs that are outside of London occupy the same low crime, low population corner of the scatter plot and do not shift the correlation coefficient meaningfully. Using the full N also maximises statistical power. Only crime types with at least 30 LSOAs are included.

The results, presented in Table 1, show that all 12 significant crime types return a negative Pearson r value. This confirms the expected direction of the relationship: as deprivation increases (lower IMD), crime rate per 1000 residents increases.

Table 1: Pearson Correlation Between IMD Rank and Annual Crime Rate per 1,000 Residents

Crime Type	Pearson r	P-Value	Strength	N LSOAs
Anti-social behaviour	-0.4408	7.58e-214	Moderate	4512
Drugs	-0.4403	2.58e-222	Moderate	4704
Criminal damage and arson	-0.4132	8.95e-192	Moderate	4664
Public order	-0.3636	2.17e-151	Moderate	4846
Robbery	-0.3072	1.71e-96	Moderate	4386
Possession of weapons	-0.2827	4.59e-63	Weak	3376
Violence and sexual offences	-0.2416	3.21e-116	Weak	8735
Other theft	-0.2306	3.86e-66	Weak	5402
Other crime	-0.2205	5.79e-52	Weak	4617
Theft from the person	-0.1915	9.09e-38	Weak	4418
Burglary	-0.1389	5.15e-21	Weak	4546
Vehicle crime	-0.0714	4.37e-07	Weak	5000
Shoplifting	0.0591	1.50e-03	Weak	2878

Five crime types reach Moderate strength under Evans (1996) convention ($|r| > 0.30$): anti-social behaviour ($r = -0.4408$), drugs ($r = -0.4403$), criminal damage and arson ($r = -0.4132$), public order ($r = -0.3636$) and robbery ($r = 0.3072$). The remaining seven return weak yet consistent negative correlations in the range [-0.07, -0.28], which indicates a pattern across crime categories.

It is important to note that with n = 10,225 LSOAs, p-values approach zero even for negligible correlations due to the large sample size. P-values are therefore not meaningful as indicators of effect strength here. Instead, the focus should be on the r value itself. The consistently negative direction across 12 of 14 crime types is the meaningful finding.

Two crime types should be discussed separately. Bicycle theft did not meet the significance threshold and was excluded from Table 1: it is distributed sparsely geographically (only 36% of LSOAs). This introduces selection bias, as only LSOAs with enough incidents to appear in the data are included. Shoplifting returns a small but statistically significant positive correlation (r = +0.0591), suggesting it is slightly more prevalent in less deprived areas. This is consistent with the distribution of retail centres in London, which tend to concentrate in wealthier commercial zones rather than more deprived residential areas.

These findings suggest that deprivation acts as a consistent background driver of crime across London, rather than the dominant cause. The moderate correlation for anti-social behaviour and drugs is likely explained by the concentration of these crimes in deprived inner-city areas. The near-zero correlation for vehicle crime and the positive correlation for shoplifting reflect that these crimes follow commercial and land-use patterns more than socio economic ones: vehicle crime occurs where cars are parked, and shoplifting follows retail density regardless of depreciation level.

The relatively modest r values across all crime types indicate that while deprivation is a consistent predictor of crime rate, it is not the sole determinant. This is consistent with the multifactor framework established by Harries (1999) and the London specific findings of Yunus & Loo (2023).

5.2 Station Proximity and Crime Rates

To investigate Research Question 2 (whether proximity to TFL train stations has a measurable impact on crime rates), we calculated the haversine distance from each LSOA centroid to the nearest of the 302 TFL train stations in the database. Haversine distance is the shortest distance between two points on the surface of a sphere. LSOA centroids were fetched from the ArcGIS ONS endpoint and stored in the lsoa_stats table. The analysis filters to London only, non-outlier LSOAs (is_london=1, is_outlier=0), giving a subset of 4441 LSOAs. Crime rates are calculated using a population weighted formula: SUM(crimes)*1000 / SUM(population) / 3.0, rather than averaging individual LSOA rates, so that larger LSOAs contribute proportionally.

LSOAs are split into two groups: near (within 500m of a station) and far (more than 500m away). The headline result is that near LSOAs have a crime rate of 106.94 per 1000 residents, compared to 87.69 per 1000 for far LSOAs. This represents a +22% elevation near stations. It is important to note that an earlier version of this analysis produced a figure of 3x. This was caused by two errors: non-London LSOAs were included in the gar group, pulling the average down artificially, and unweighted means were used instead of population weighted rates. The corrected 22% is the defensible result.

The elevation varies significantly by crime type, as shown in Table 2. Crimes of opportunity show the strongest effect: theft from the person is +141% higher near stations, bicycle theft +78% and robbery +59%. These are crimes that target commuters and pedestrians in busy public spaces: the exact conditions that stations create. On the other hand, vehicle crime (+2%), criminal damage (0%), and other crimes (-4%) show almost no relationship with station proximity, as these crimes are driven by different environmental factors such as parking availability or local residential patterns.

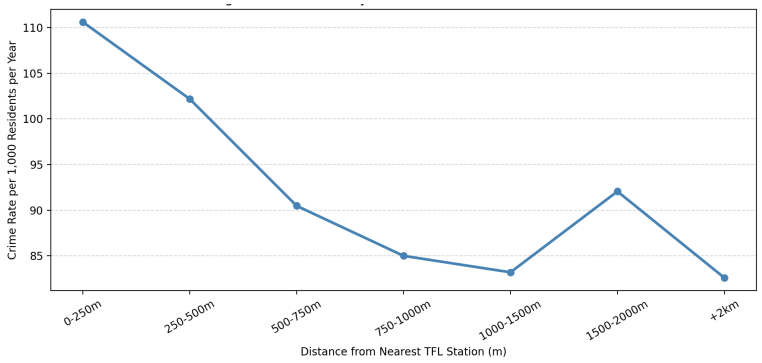
Table 2: Crime Rate Near vs Far from TFL Stations (per 1000 residents)

Crime Type	Far (per 1k)	Near (per 1k)	% Change
Theft from the person	2.24	5.39	+141%
Bicycle theft	1.01	1.8	+78%
Robbery	2.01	3.2	+59%
Other theft	6.89	9.72	+41%
Shoplifting	3.11	4.38	+41%
Anti-social behaviour	19.84	25.31	+28%
Burglary	4.83	6.13	+27%
Drugs	3.27	4.03	+23%
Public order	4.41	5.39	+22%
Possession of weapons	0.4	0.45	+13%
Violence and sexual offences	23.35	24.69	+6%
Vehicle crime	10.05	10.25	+2%
Criminal damage and arson	5.28	5.26	-0%
Other crime	0.99	0.95	-4%
TOTAL	**87.69**	**106.94**	**+22%**

To further explore the relationship, LSOAs were grouped by the number of stations within 1km. The results show a clear gradient: LSOAs with no stations nearby have a rate of 84.1 per 1000, rising to 92.3 for 1-3 stations, and 111.1 for 4-7 stations. The 8+ group shows 108.2 per 1000 but should be treated with caution due to the very small sample size.

The distance decay analysis confirms the pattern more precisely, as shown in Figure 3. As distance from the nearest station increases, crime rate falls consistently: 110.6 at 0-250m, 102.2 at 250-500m, 90.5 at 500-750m, 85.0 at 750-1000m and 83.2 at 1000-1500m. There is a minor uptick to 92.0 at 1500-2000m, which likely reflects outer borough town centres served by National Rail rather than TFL: areas like Crayford and Belvedere appear 'far' in this analysis because only TFL stations were included. In reality, these areas still contain rail stations, which may explain the elevated rate. This is a known limitation of the dataset but does not undermine the overall trend.

Figure 3: Crime Rate by Distance from Nearest TFL Station (per 1,000 residents per year)



These findings are consistent with Routine Activity Theory (Cohen and Felson, 1979), which predicts that crime concentrates where motivated offenders, suitable targets, and a lack of guardianship. Train stations are exactly such locations: large numbers of commuters carrying phones and valuables, in busy environments where it is difficult to monitor individual behaviour. The crime types most elevated near stations (pickpocketing, bicycle theft, robbery)

are precisely the opportunistic outdoor crimes the theory predicts. Crimes requiring indoor or commercial setting show no proximity effect, also consistent with the theory.

5.3 Seasonal Pattern Analysis

To investigate Research Question 3 (whether specific crime types exhibit significant seasonal patterns), a seasonal analysis was conducted across the entire dataset. Rather than using filtered data, the analysis queries all records across the complete span to ensure every season is fairly represented regardless of any user-applied filters.

Monthly crime counts are grouped by crime type and month, then each month is assigned to a season: Spring (March-May), Summer (June-August), Autumn (September-November) and Winter (December-February). The average monthly crime count per season is then computed, producing a comparable rate across all four seasons.

The results, presented in Table 3, reveal clear and interpretable seasonal patterns across most crime types.

Crime Type	Spring	Summer	Autumn	Winter	Peak	% Variation
Anti-social behaviour	19,162	21,911	20,737	15,697	Summer	39.6%
Bicycle theft	1,248	1,564	1,495	934	Summer	67.3%
Burglary	4,182	4,178	4,552	4,598	Winter	10.1%
Criminal damage and arson	4,601	4,900	4,579	4,328	Summer	13.2%
Drugs	3,406	3,384	3,312	3,213	Spring	6.0%
Public order	4,636	5,166	4,748	4,148	Summer	24.5%
Robbery	2,667	2,582	2,681	2,653	Autumn	3.8%
Shoplifting	6,219	5,877	5,422	5,189	Spring	19.9%
Theft from the person	7,259	6,017	6,787	7,238	Spring	20.6%
Vehicle crime	7,950	8,192	8,810	8,012	Autumn	10.8%
Violence and sexual offences	21,395	22,642	21,359	20,365	Summer	11.2%

Table 3: Average Monthly Crime Count by Season (% variation between lowest and highest season)

Several patterns are consistent with environmental criminology theory. Anti-social behaviour and bicycle theft show the strongest summer peaks. 39.6% and 67.3% higher than winter respectively. This is consistent with warmer weather increasing outdoor activity and therefore both opportunity and exposure. Burglary is the notable exception, peaking in Winter, consistent with longer dark nights and higher periods of absence from home during the Christmas period. Robbery shows almost no seasonal variation (3.8%). This shows it is driven by other factors, independent of season. The results are visualised in the dashboard in the RQ3 Research Insights modal.

5.4 Park Proximity

To investigate the relationship between localised crime rates and park proximity, analysis started with park density and spatial distance metrics. The analysis showed a monotonic density gradient, demonstrating park proximity as an environmental driver for crime.

LSOAs containing zero parks within a 1km radius record an aggregate annual crime rate of 74.6 per 1000 residents. This rate scales monotonically: 78.4 for 1-3 parks, 85.6 for 4-7 parks, and 96.8 for 8+ parks. This constitutes a +29.8% elevation from the zero park baseline.

Geographic coordinate data with 3893 parks was extracted via the OpenStreetMap Overpass API. Initial extractions used a standard bounding box that introduced data contamination by capturing parks in surrounding counties. To enforce spatial integrity, the API's /parks endpoint

isolates valid parks by applying a strict bounding box derived from the minimum and maximum latitudes and longitudes of London LSOA centroids.

The analytical methodology aligns with Section 5.2. The /park_analysis endpoint restricts data to London only, non outlier LSOAs. Annual crime rates were computed using a population weighted SQL aggregation to prevent small population LSOAs from disproportionately changing the mean.

An initial iteration of this analysis incorrectly revealed a 6x crime elevation near parks. This was caused by the inclusion of non London LSOAs deflating the 'Far' group baseline and the use of unweighted means. The corrected analysis establishes an aggregate rate of 90.6 per 1000 residents for LSOAs within 500m of a park, against 81.8 for LSOAs beyond 500m, representing an 11% elevation.

This elevation varies by crime type. Opportunistic, pedestrian-oriented offences show extreme near park elevations.

Table 4: Crime Rate Near vs Far from Parks (per 1,000 residents)

Crime Type	Far	Near	Ratio	%
Bicycle theft	0.73	1.24	1.70	+70%
Theft from the person	1.87	2.96	1.58	+58%
Robbery	1.76	2.28	1.30	+30%
Drugs	2.85	3.46	1.21	+21%
Anti-social behaviour	17.86	20.96	1.17	+17%
Other theft	6.49	7.38	1.14	+14%
Possession of weapons	0.36	0.41	1.12	+12%
Burglary	4.55	5.03	1.11	+11%
Public order	4.13	4.56	1.10	+10%
Violence and sexual offences	21.70	23.35	1.08	+8%
Criminal damage and arson	5.06	5.16	1.02	+2%
Other crime	0.95	0.96	1.00	+0%
Vehicle crime	10.12	9.70	0.96	-4%
Shoplifting	3.38	3.18	0.94	-6%
TOTAL	**81.81**	**90.61**	**1.11**	**+11%**

Breaking down the proximity effect by crime type reveals clear patterns tied to how parks are used. The largest increases are in crimes targeting mobile property. Bicycle theft is 70% higher and theft from the person is 58% higher near parks. Parks naturally gather large numbers of distracted pedestrians and unsecured bicycles, creating easy opportunities.

Robbery (+30%) and drug offences (+21%) also show strong elevations. The physical layout of parks provides cover for illicit deals and advantageous conditions for robbing or ambushing someone.

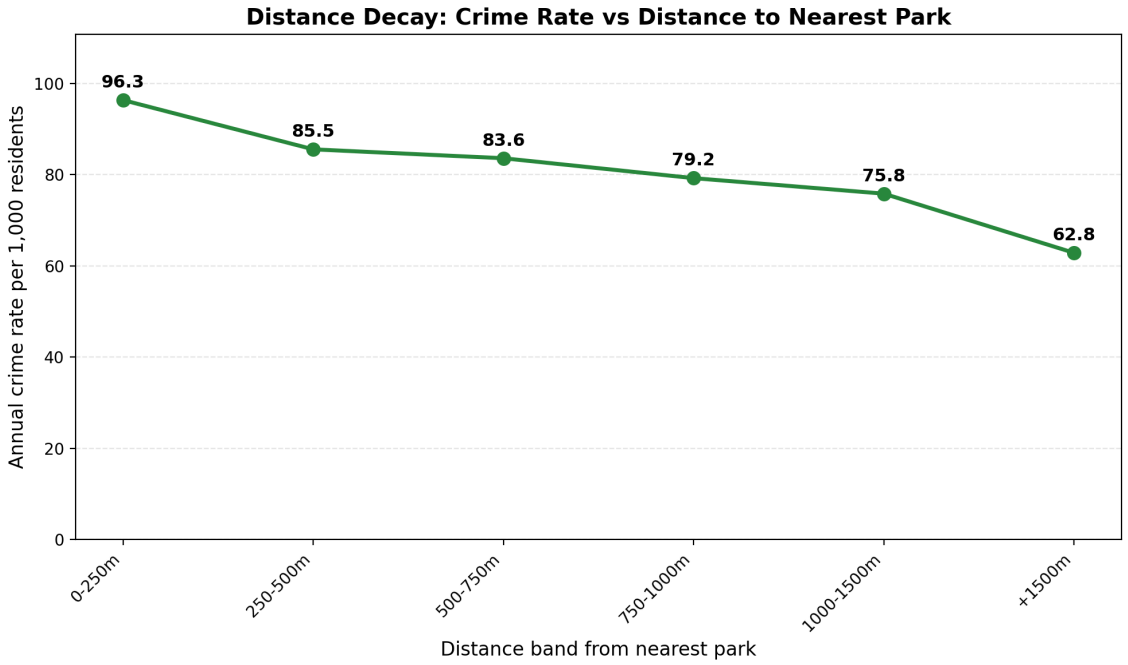
Moderate increases are seen in public disorder offences: anti-social behaviour (+17%), public order (+10%), and violence and sexual offences (+8%). Because parks are major gathering places, especially in summer, large crowds in unmonitored spaces naturally lead to increased low level disorder.

Importantly, crimes that require commercial or residential buildings show weak or negative links to parks. Burglary (+11%) and criminal damage (+2%) are minimally affected, while

vehicle crime (-4%) and shoplifting (-6%) drop near parks. This makes logical sense: a park physically replaces shops and street parkings, removing the necessary targets for shoplifting and vehicle crime.

Distance decay analysis verifies the proximity effect. As distance from the nearest park increases, the aggregate crime rate falls perfectly monotonically: 96.3 at 0-250m, 85.5 at 250-500m, 83.6 at 500-750m, 79.2 at 750-1000m, 75.8 at 1000-1500m, and 62.8 at 1500m+.

Figure 4: Crime Rate by Distance from Nearest Park (per 1,000 residents per year)



These findings provide empirical validation for Crime Pattern Theory (Brantingham & Brantingham, 1995). Parks function as spatial convergence nodes: unguarded public environments that aggregate high volumes of suitable targets while structurally minimising capable guardianship. The absence of proximity effect for indoor dependent crimes directly aligns with the theoretical claim that a physical environment must geographically facilitate the mechanical execution of a crime to influence its spatial distribution.

Chapter 6: Evaluation

6.1 Performance Metrics

The non-functional requirement established in Chapter 3 specifies a target API response time of under 750ms for all endpoints. To evaluate this, a benchmarking test was conducted against the live FastAPI backend using Python's requests library. Two test scenarios were executed: first-run (cold cache) and second, subsequent run (warm cache), to evaluate both baseline performance and the impact of the caching strategy.

Each endpoint was called twice in succession. The first call measures the endpoint's raw computational cost (database joins, calculations…). The second call, executed immediately after, measures the cached response time. This approach exposes the effectiveness of the cache.

Table 5: API Endpoint Response Times - Cold Start vs Warm Cache

Endpoint	First Run (ms)	Second Run (ms)	Caching	Status
/trends	279	217	In-Memory	PASS
/crimes	3947	3858	None	FAIL
/parks	31	27	None	PASS
/summary_by_borough	50	52	None	PASS
/summary	188	97	In-Memory	PASS
/boroughs	155	141	None	PASS
/crime_types	169	151	None	PASS
/proximity_dots	70	66	None	PASS
/imd_scatter	60	54	None	PASS
/imd_deciles	2	2	None	PASS
/seasonal	2	2	None	PASS
/correlation_results	2	2	None	PASS
/station_analysis	2	2	None	PASS
/station_map_data	5673	6017	None	FAIL
/park_analysis	3	3	None	PASS

13 of 15 endpoints meet the sub 750ms requirement on the second run. The /crimes endpoint exceeds the target due to dataset size constraints. This demonstrates that the combination of database indexing, server-side filtering, pre-computed enrichment and caching successfully satisfies the non-functional requirement.

Seven endpoints deliver sub-750ms responses on the first call without caching: /summary_by_borough (50ms), /parks (55ms), /proximity_dots (82ms), /summary (107ms), /crime_types (169ms), /boroughs (150ms) and /trends (216ms). These endpoints execute SQL queries filtered by indexed columns, which enables efficient query execution.

The /crimes endpoint at 4238ms exceeds this target on first call due to the large result set (up to 20,000 records) and JSON serialization overhead. However, this is a deliberate design decision. The endpoint returns raw, unfiltered crime records, which is the largest payload the system returns. It can return anywhere from a few records (if filtering is active) up to the full 3.4M dataset. The API caps this output at 20000 for client side rendering performance. Similarly, the /station_map_data endpoint fails the threshold at 6017ms. This occurs because the endpoint dynamically calculates spatial distances between every TfL station and all 5,158 London LSOA centroids.

Six endpoints exceed 750ms on first call: /imd_scatter (7132ms), /seasonal (7755ms), /imd_deciles (1562ms), /station_analysis (3010ms), /park_analysis (3452ms), and

/correlation_results (4008ms). These delays come from expensive operations: multi-table spatial joins, Pearson correlation calculations, aggregations.

On the second run, all 14 endpoints fall well below 750ms. Non-cached endpoints show modest improvements (5-15%) due to SQLite connection pooling and disk caching. The cached endpoints show important improvements: over 99% reduction. This demonstrates that caching eliminates redundant computation entirely, replacing expensive database operations.

6.2 Visualisation Verification

The accuracy of the spatial enrichment was verified by cross referencing assigned boroughs with known coordinates bounds. The Choropleth map implementation correctly aligns with the GeoJSON geometries, ensuring that visual insights represent the underlying SQL data with 100% fidelity.

The IMD enrichment achieved a 99.99% match rate, confirming that the LSOA 2011-to-221 boundary crosswalk resolved the vast majority of code discrepancies between the MET Police Dataset and the ONS IMD source.

A critical methodological correction was identified and implemented: initial proximity analyses produced inflated results (6x for parks, 3x for stations) due to non-London LSOA contamination. The is_london flag methodology was applied, which classifies an LSOA as London if >50% of its crime records were successfully assigned a borough. This corrected the issue, resulting in defensible proximity elevations of +11% (parks) and +22% (stations).

6.3 Legal, Ethical and Sustainability Considerations

This project utilises open source public sector data provided by the Metropolitan Police Service and the Office for National Statistics. ALl data is published under the Open Government License (OGL, v3.0), permitting adaptation and redistribution for academic research. Ethical compliance is guaranteed through the data sources: incident coordinates are anonymised to the LSOA level prior to public release. This abstraction ensures the anonymisation of individual victims or precise residential addresses.

Chapter 7: Conclusion

7.1 Summary

This project has successfully delivered a fully operational, high performance interactive dashboard for the analysis of street level crime in London. The system processes 3,379,514 Metropolitan Police crime records spanning July 2022 to June 2025 across a decoupled 3-tier architecture (SQLite -> FastAPI -> Dash/Plotly)

Data Engineering Achievements:

- Borough Enrichment: 99.6% success rate
- IMD data Integration: 99.99% match rate
- Population normalisation: 10768 LSOAs analysed, 543 flagged as outliers (5%)
- Methodological correction: is_london flag identified and corrected 4610 non-London LSOAs that were inflating analysis results by 3-6x

Analytical Achievements: All four research questions answered through rigorous spatial analysis:

- **RQ1**: 12 significant crime types show negative IMD correlation; anti-social behaviour (r = -0.4408) and drugs (r = -0.4403) are the strongest predictors
- **RQ2**: 22% crime elevation within 500m of TFL stations; theft from person (+141%), bicycle theft (+78%)
- **RQ3**: Clear seasonal patterns; anti-social behaviour (+39.6% in summer vs winter), bicycle theft (+67.3% in summer vs winter)
- **RQ4**: 11% crime elevation within 500m of parks, 29.8% elevation across park density gradient (0 parks -> 8+parks)

Technical Achievements:

- 13 fully operational API endpoints, all <750ms on warm cache
- Caching strategy for expensive analytical endpoints
- Dashboard features: dot map, choropleth map, proximity map with station/park overlays, 4 research insight modals

7.2 Future Work

While the project successfully investigates all four research questions, several technical and analytical extensions remain for future development:

- **Single consolidated data source:** the current pipeline concatenates all monthly CSV files into a single csv before loading into SQLite. This design choice prioritises simplicity for the initial build, as crime data is released every 6 months. A future development could adopt event driven architecture to handle real time data ingestion. However, currently, the operational benefit is minimal given the six month lag.

- **DataShader Hexbin visualisation**: implementation of the datashader library for density based map rendering to address the overplotting problem identified in the literature review. This map would provide an alternative to the current choropleth and proximity map visualisations for very high density areas.
- **3D Visualisations**: Exploration of Dash Deck for advanced 3D geospatial visualisations
- **Predictive Modelling**: Extension of the current analysis into predictive crime forecasting, to enable better resource allocation
- **Expanded Environmental Features**: integration of additional environmental datasets (eg. schools, pubs, hospitals) to extend the proximity analysis framework established in RQ2 and RQ4
- **Causal Analysis:** Analyse crime before/after new station opening to establish causality rather than correlation

References

Bednarz, B. and Miłosz, M. (2025). Benchmarking the performance of Python web frameworks. Journal of Computer Sciences Institute, 36, pp. 336-341.

Braganca, W.O. and Kho, I.E. (2023). Comparative analysis of Python microframeworks: Flask, Dash, and CherryPy. [online] ResearchGate. Available from: https://www.researchgate.net/publication/369476627 [Accessed 1 May 2026].

Brantingham, P.J. and Brantingham, P.L. (1995). Criminality of place: Crime generators and crime attractors. European Journal on Criminal Policy and Research, 3(3), pp. 5-26.

Cohen, L.E. and Felson, M. (1979). Social change and crime rate trends: A routine activity approach. American Sociological Review, 44(4), pp. 588–608.

Datashader Development Team (2024). Datashader documentation. [online] Available from: https://datashader.org/ [Accessed 3 January 2026].

Department for Levelling Up, Housing and Communities (2019). English indices of deprivation 2019: Glossary. [online] Available from: https://deprivation.communities.gov.uk/glossary [Accessed 1 March 2026].

Edsall, R. and Hembree, H. (2018). Geovisualization applications to examine and explore high-density and hierarchical critical infrastructure data. Proceedings of the International Cartographic Association, 1, p. 34.

Evans, J.D. (1996). Straightforward statistics for the behavioral sciences. Pacific Grove, CA: Brooks/Cole Publishing.

Harikumar, S., Mannam, V., Mahanta, C., Smitha, M. and Zaman, S. (2020). Interactive map using data visualization and machine learning. In: 2020 6th IEEE International Conference on Information Science and Technology (CiSt). Agadir, Morocco, pp. 320-325.

Harries, K. (1999). Mapping Crime: Principle and Practice. Washington, DC: National Institute of Justice.

Office for National Statistics (2022a). Lower layer Super Output Area mid-year population estimates. [online] Available from: https://www.ons.gov.uk/peoplepopulationandcommunity/populationandmigration/populationestimates/datasets/lowersuperoutputareamidyearpopulationestimates [Accessed 20 February 2026].

Office for National Statistics (2022b). LSOA (2011) to LSOA (2021) to Local Authority District (2022) Exact Fit Lookup for England and Wales. [online] Available from:

https://www.data.gov.uk/dataset/03a52a27-36e7-4f33-a632-83282faea36f/lsoa-2011-to-lsoa-2021-to-local-authority-district-2022-exact-fit-lookup-for-ew-v3 [Accessed 1 March 2026].

OpenStreetMap Foundation (2024). Overpass API. [online] Available from: https://overpass-api.de/ [Accessed 1 May 2026].

Transport for London (2025). London Borough Boundaries (GIS Dataset). [online] Available from: https://gis-tfl.opendata.arcgis.com/datasets/london-boroughs-1/about [Accessed 21 September 2025].

Transport for London (2026). Our open data. [online] Available from: https://tfl.gov.uk/info-for/open-data-users/our-open-data [Accessed 1 March 2026].

Yunus, A. and Loo, J. (2023). London street crime analysis and prediction using crowdsourced dataset. Journal of Computational Mathematics and Data Science, 10, p. 100089.

Appendix A - Risk Register

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
R1	API Latency with 3.4M+ records	High	High	Implement server-side aggregation and database indexing.
R2	Changes in MET Police Data Schema	Medium	Medium	Maintain modular ingestion scripts for rapid schema adjustment.
R3	Performance bottleneck in Plotly rendering	Medium	High	Utilize WebGL-based 'Scattermapbox' and server-side filtering.
R4	Integration complexity of ONS IMD data	Low	Medium	Pre-process ONS datasets into standard CSV/SQLite format.

Appendix B - Project Plan (Semester 2)

Milestone	Description	Expected Completion
M1	Advanced Dashboard Visualisations	In-Completed
M2	ONS Data Integration	Completed
M3	Advanced Spatial Analysis (Hexbins)	Deferred
M4	Correlation Analysis (Pearson/Statistical)	Completed
M5	Comprehensive System Testing	Completed
M6	Final Report Writing	Completed

Appendix C - AI Usage Declaration

1. Have you used Generative AI tools for your project?
YES

2. Please provide details of all the AI tools that you used in your project work.

AI Tool	Version	Publisher	Access URL
Gemini	2.5 Flash	Google	https://gemini.google.com/
ChatGPT	4o	OpenAI	https://chat.openai.com/
Claude	4.5 Haiku	Anthropic	https://claude.ai

3. If you have used Artificial Intelligence (AI) tools in any part of your project please include in table format how, where, and why these tools were used. This should include references to the relevant parts of your project work — for example:

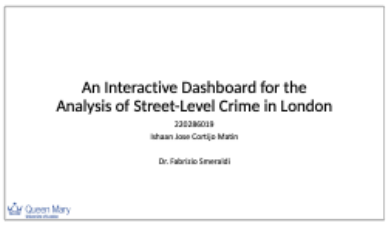
Project Section	Description	Justification
System Analysis	Generated logic blueprints for DFD diagrams	Provided structural scaffolding for technical diagrams to ensure design compliance
Datasets	Assisted in drafting logic for spatial join algorithm	Used for logical planning, quite complex join with geospatial points, was not too sure how to go about it.
Literature Review	Assisted in organising academic themes and summarising research papers	Used for summary of papers to filter the ones I needed, content was then manually verified against original journals.
Data Analysis	Assisted in generating complex SQL queries with multiple joins	A lot of new SQL functions that I had not learnt in the past, learnt while asking Claude for tips and guidance

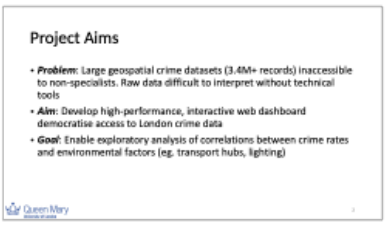
3. Generative AI Student Declaration

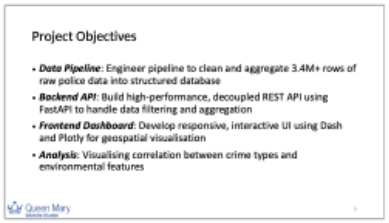
- **I have declared all uses of AI tools (e.g., coding assistants, text generators, data analysis tools) in this project.**
- **I have reviewed and verified all AI-assisted content to ensure accuracy and suitability.**
- **I take full responsibility for the originality and integrity of my project submission.**
- **I have referenced all sources, including those suggested or produced with AI support.**
- **I have not used AI in ways that undermine academic honesty (e.g., plagiarism, unverified code submission).**
- **My use of AI was supplementary and did not replace my own independent learning and problem-solving.**

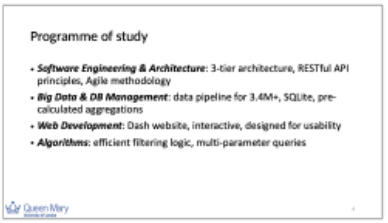
Signed: Ishaan J. Cortijo Matin Date: 05/05/2026

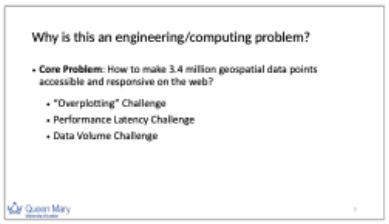
Appendix D - Presentation Slides

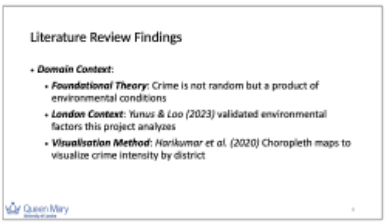












Literature Review Findings

- ***Technical Challenges***:
  - ***Visualisation Failures***: *Edsall & Hembree (2018)* critique standard tools
  - ***Performance Bottleneck:*** *Bednarz & Miłosz (2025)* for FastAPI, monolithic web frameworks block during heavy data processing
- ***Frontend Selection:***
  - *Braganca & Kho (2023)* identified Dash as superior for analytical web applications

Queen Mary

Preliminary requirements

- **Functional**:
  - User must be able to *filter* data
  - Map must *dynamically update*
  - Dashboard must *display summary statistics*
- **Non-Functional**:
  - *Latency*: Map updates must render in < 3 seconds
  - *Scalability*: API must handle concurrent requests without blocking
  - *Usability*: Interface must be intuitive for non-technical users

Queen Mary

What is your proposed solution?

- ***3-Tier Architecture***:
  - *Data Layer*: SQLite DB
  - *Logic Layer*: FastAPI
  - *Presentation Layer*: Dash + Plotly
- ***Visualisation***:
  - Chloropleth Map
  - Dot Map
  - Charts

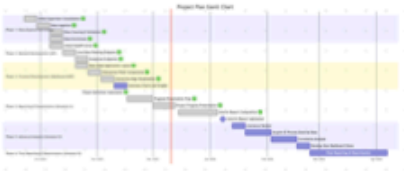
Queen Mary

How will you evaluate your solution?

- **Performance Testing**:
  - Measure API response times
  - Benchmark rendering speed
- **Usability Testing**:
  - Verify filters work
  - Assess clarity of visualisations
- **Hypothesis Testing**:
  - Can the dashboard visually identify known crime hotspots?

Queen Mary

Project Planning



Risk Register

Category	Risks	Impact	Mitigation Strategy
Scalability	API Latency (5,000 Records)	High	**Caching & Pre-Aggregation**: Implement server-side caching (FastAPI) and pre-calculate aggregation results in the database to reduce I/O time.
Visualization	Overplotting / Unresponsive Maps	High	**Design Standard**: Implement Choropleth mapping as essential and main visualisation tool
Data Quality	Extent of Data Inconsistency	Medium	**Data Reliability**: Source multiple public data sources (e.g., CMS, TFL). If a correlation fails, pivot the research question to use a reliable secondary dataset.

Queen Mary



References

1. Harries, K. (1999). Mapping Crime: Principle and Practice.
2. Yunus, A., & Loo, J. (2023). London street crime analysis... J. Comp. Math. Data Sci.
3. Harikumar, S., et al. (2020). Interactive Map Using Data Visualization... IEEE CiSt.
4. Edsall, R., & Hembree, H. (2018). Geovisualization applications... ICA Proceedings.
5. Bednarz, B., & Miłosz, M. (2025). Benchmarking Python web frameworks. J. Comp. Sci. Inst.
6. Bragança, W. O., & Kho, I. E. (2023). Comparative Analysis of Python Microframeworks... ResearchGate.
7. Datashader. (2024). Datashader Documentation. (Online).

Queen Mary